



南方科技大学

LAB 4: Recurrent Neural Networks

COE 201 - AI Large Models, Spring 2026

2026-05-17

College of Engineering, SUSTech

Office Hour: Thursday, 14:30-15:30 @ 3rd Research Building, Room 610

Outline

- Sequence Modeling Motivation 1
- Understanding RNN 4
- Implementation in PyTorch 10
- Lab Tasks 16

Sequence Modeling Motivation

The Problem with Feedforward Networks



南方科技大学

Problem: Standard feed-forward neural networks treat each input independently. But many real-world tasks involve **sequential data**:

- Text: “I love this movie”
- Time series data: Stock prices, sensor data

Example1 (text): “The movie was not good” vs “The movie was good, not bad”

- Same words, different meaning - **order matters!**

Example2 (stock price):



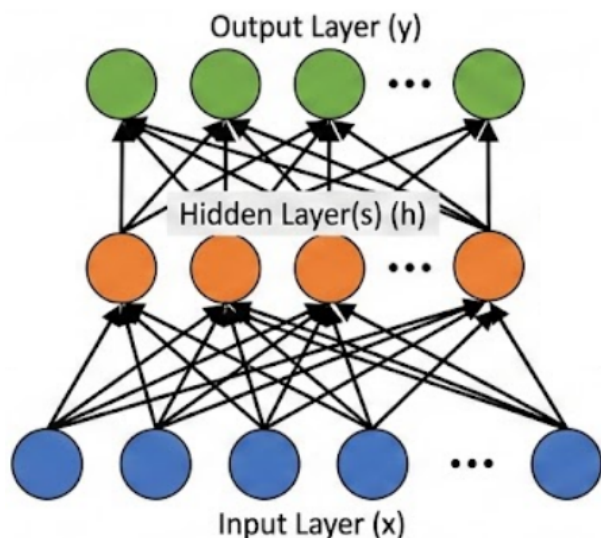
Feedforward vs. Recurrent



Solution: A network that can **remember** previous inputs (**need memory!**).

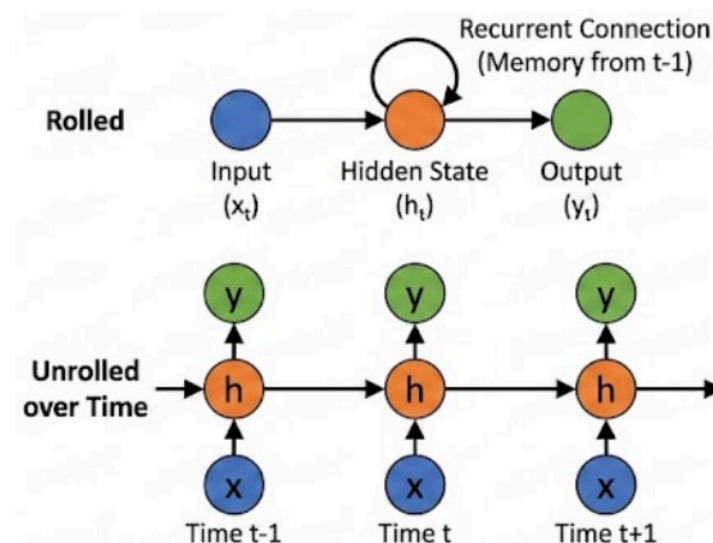
Feedforward NN:

- Each input processed independently
- No memory of past inputs



Recurrent NN:

- Maintains a **hidden state** carries information from previous steps



Understanding RNN

RNN Architecture: The RNN Cell



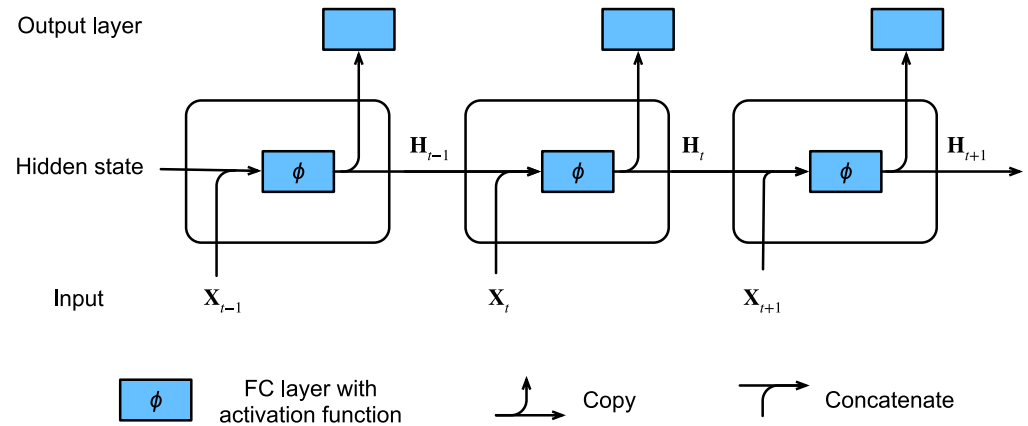
南方科技大学

An RNN processes sequences one step at a time, maintaining a **hidden state**:

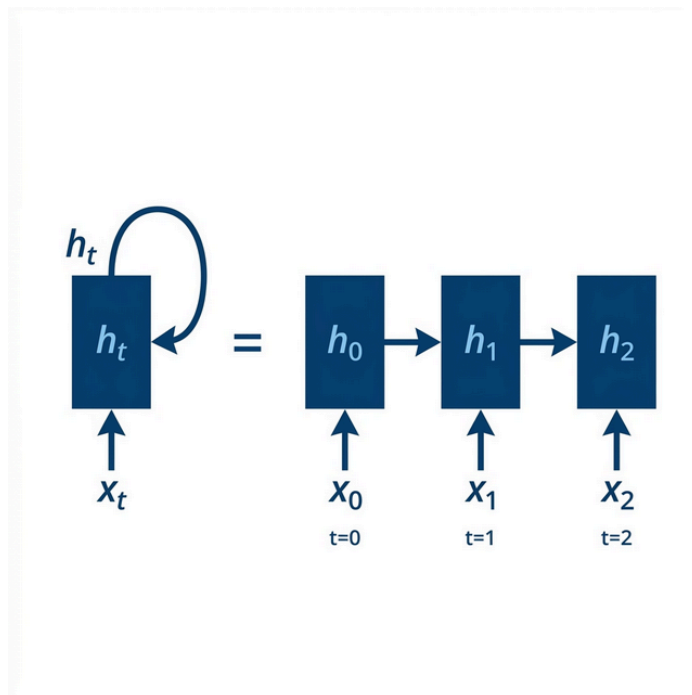
$$H_t = \tanh(W_h H_{t-1} + W_x x_t + b)$$

Where:

- x_t : input at time step t
- H_{t-1} : hidden state from previous step
- H_t : new hidden state (output)
- W_h, W_x, b : learnable parameters



An RNN unrolled in time is just a deep network with **shared weights**:



Key insight: The same parameters are used at every time step!

The Hidden State as “Summary”



Think of the **hidden state** H_t as a “running summary” of the sequence.

The Mental Model:

- H_0 : “I’ve seen nothing yet”
- H_1 : “I’ve seen ‘The’”
- H_2 : “I’ve seen ‘The cat’”
- H_t : “I’ve seen everything from step 0 to t ”

Information Accumulation: At each step, we **blend** the new input (x_t) with our previous memory ($H_{\{t-1\}}$).

```
1 # Conceptual update
2 memory = 0
3 for word in sentence:
4     memory = combine(memory, word)
5
6 # The final 'memory' holds
7 # the context of the
8 # entire sentence!
```

The

Example: "I grew up in France... [100 words] ... I speak fluent [?]"

To predict “**French**”, the model must remember “**France**” from 100 steps ago.

1. Vanishing Gradients

- Gradients multiplied by W_h every step.
- If W_h is small, the signal “fades”.
- **Intuition:** The model **forgets** “France” by the time it reaches “French”.

2. Exploding Gradients

- If W_h is large, gradients grow exponentially.
- **Result:** Parameters explode (NaN).
- **Solution: Gradient Clipping.**

Standard RNNs struggle with long-term memory due to vanishing gradients.

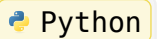
Modern Gated RNNs:

- **LSTM** (Long Short-Term Memory)
- **GRU** (Gated Recurrent Unit)
- Use “Gates” to protect memory, allowing gradients to flow over hundreds of steps.

PyTorch Implementation:

One-line drop-in replacement!

```
1 # Just replace RNN with LSTM
2 self.rnn = nn.LSTM(embed_dim, hidden_dim,
3                     batch_first=True)
```



While LSTM/GRU improved things, they are still **sequential** (slow $O(N)$ for-loop).

- **This Course:** We won't dwell on the complex internal gates of LSTM/GRU.
- **Next Step:** We will soon introduce **Transformers**.

Implementation in PyTorch

Sequence Tensor Shapes



Critical tensor shapes when processing sequences in PyTorch:

Data Component	Shape	Meaning
Input Data	(Batch, Seq, Feat)	Batch of sequences
Hidden State	(Batch, Hidden)	Memory at a specific time step
Model Output	(Batch, Seq, Vocab)	Output distribution at each step

```
1 # Example: Batch of 16 text snippets, each 25 characters long.
2 # We embed each character into a 32-dim vector.
3 x_embedded = torch.randn(16, 25, 32) # (Batch, Seq, Feat)
```

 Python

Using nn.RNN: Initialization & Forward



```
1 # 1. Initialization (__init__)
2 # input_size: Dimension of each element in the sequence (e.g., embed_dim)
3 # hidden_size: Dimension of the memory/hidden state
4 # batch_first: If True, input shape is (Batch, Seq, Feature)
5 rnn = nn.RNN(input_size=10, hidden_size=20, batch_first=True)
6
7 # 2. The Forward Pass
8 # x shape: (Batch, Seq_Len, Input_Size)
9 # out shape: (Batch, Seq_Len, Hidden_Size) -> Hidden state for EVERY step
10 # hn shape: (Num_Layers, Batch, Hidden_Size) -> Hidden state for LAST step only
11 out, hn = rnn(x)
```

Python

Tensor	What it represents
out	The “history” of hidden states across all time steps.
hn	The final “summary” used for prediction/classification.

Under the Hood: The Recurrence Loop



How does `nn.RNN` work internally? It's just a **for-loop** over the sequence!

```
1  # Conceptual PyTorch Implementation
2  def forward(x, h_0):
3      # x: (Batch, Seq_Len, Dim)
4      outputs = []
5      h_t = h_0
6      for t in range(seq_len):
7          x_t = x[:, t, :] # Current step
8          h_t = torch.tanh(W_ih @ x_t + b_ih + W_hh @ h_t + b_hh)
9          outputs.append(h_t)
10
11     return torch.stack(outputs, dim=1), h_t
```



Building a Sentiment Classifier



In classification, we typically “squeeze” the sequence into a single vector using the **last** hidden state.

```
1  class RNNClassifier(nn.Module):
2      def __init__(self, vocab_size, embed_dim, hidden_dim):
3          super().__init__()
4          self.embedding = nn.Embedding(vocab_size, embed_dim)
5          self.rnn       = nn.RNN(embed_dim, hidden_dim, batch_first=True)
6          self.fc        = nn.Linear(hidden_dim, 2)
7      def forward(self, x):
8          # 1. Embed words: (B, S) -> (B, S, E)
9          x = self.embedding(x)
10         # 2. Process sequence: Return (out, hn)
11         _, hn = self.rnn(x)
12         # 3. Predict using LAST hidden state: (1, B, H) -> (B, H)
13         return self.fc(hn.squeeze(0))
```

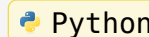
Python

Data Prep: Why Padding?



Sentences have different lengths, but PyTorch requires **rectangular tensors** for batching.

```
1 import torch
2 from torch.nn.utils.rnn import pad_sequence
3
4 sequences = [torch.tensor([1, 2, 3]), torch.tensor([4, 5])]
5 # Pad to max length (3)
6 padded = pad_sequence(sequences, batch_first=True, padding_value=0)
7 # Result: [[1, 2, 3], [4, 5, 0]]
```



Row	Sequence
1	"The movie was great"
2	"Terrible movie! <PAD> <PAD>"

Lab Tasks

Complete the exercises in `lab4/`.

The **Stanford Sentiment Treebank** (SST-2) is the industry-standard benchmark for **Binary Sentiment Classification**.

- Sentiment Analysis (detecting if text is happy/sad, positive/negative).
- **Task Type**: Classification (input text and output whether it is positive or negative).
- **Source**: Movie reviews from Rotten Tomatoes.
- **Labels**: 0 = Negative, 1 = Positive.
- **Size**: 67,000 sentences.

Sentence	Label
"a visually stunning masterpiece"	1
"a complete waste of time"	0

Task 1: Warm-up - Sequence Shapes (`lab4_warmup.py`) [40 points]

- Complete the shape verification in the script.
- Understand the relationship between `output` and `hn`.

Task 2: RNN Sentiment Classifier (`lab4_rnn.py`) [60 points]

- Complete the `SentimentRNN` class and the `forward` pass.
- Understand padding implementation and the use of **Pre-padding**.

Guide: Implementing GRU (Bonus)



南方科技大学

GRU is a variant of **RNN** that uses **gates** to control the flow of information.

Refer to: https://zh.d2l.ai/chapter_recurrent-modern/gru.html

The mathematical equations for `CustomGRUCell`:

- Gates:** $r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r)$ (Reset Gate)
 $z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z)$ (Update Gate)
- Candidate & State Update:** $\tilde{h}_t = \tanh(W \cdot [r_t \cdot h_{t-1}, x_t] + b)$ (Candidate)
 $h_t = (1 - z_t) \cdot h_{t-1} + z_t \cdot \tilde{h}_t$ (Final)

Task 3 (Optional): GRU Implementation (`lab4_gru.py`) [Bonus]

- Complete the `CustomGRUCell` and `SentimentGRU`.
- Evaluate performance on **Long** vs. **Short** reviews.

Guide: Implementing GRU (Bonus)



南方科技大学

- Analyze how GRU handles different sequence lengths.

Thanks for Listening!